

25 Kubernetes Production Issues Every DevOps Engineer Should Know

A practical, hands-on production troubleshooting guide for DevOps and Platform Engineers — root causes, exact `kubectl` commands, and concrete fixes for the 25 most common Kubernetes incidents.

→ Connect on LinkedIn: <https://www.linkedin.com/in/shubham-shirbhate-7b997715a/>

→ Subscribe on YouTube: <https://www.youtube.com/@shubhamshirbhate5366>

Table of Contents

1. CrashLoopBackOff
2. ImagePullBackOff
3. OOMKilled
4. Pending Pods
5. Node Not Ready
6. Readiness Probe Failed
7. Liveness Probe Failed
8. DNS Resolution Failure
9. PVC Pending
10. PVC Full
11. High CPU Usage
12. High Memory Usage
13. Service Not Reachable
14. Ingress Not Working
15. Certificate Expired
16. Secrets Not Mounted

17. ConfigMap Changes Not Reflected
18. Deployment Stuck
19. NetworkPolicy Blocking Traffic
20. HPA Not Scaling
21. etcd Performance Issue
22. API Server Slow
23. Worker Node Disk Full
24. Failed Rolling Update
25. Namespace Stuck in Terminating

CrashLoopBackOff

Overview

CrashLoopBackOff means the container starts, exits (crashes or completes), and kubelet keeps restarting it with an exponentially increasing back-off delay (10s, 20s, 40s ... capped at 5 min). It is a symptom, not a root cause — the real cause is almost always inside the application, its config, or its startup dependencies.

Symptoms

- `kubectl get pods` shows STATUS = CrashLoopBackOff and RESTARTS climbing.
- Pod cycles between Running and Error/Completed.
- Service using the pod intermittently loses backends.

Root Causes

- Application throws an unhandled exception on startup or exits with a non-zero code.
- Missing or malformed environment variables / config files the app requires at boot.
- A dependency (database, downstream API, cache) is unreachable and the app has no retry logic.
- Wrong container command/entrypoint or a missing binary in the image.
- Liveness probe killing the container before the app finishes initializing.
- OOM kill disguised as a crash (check exit code 137, covered in issue 3).

Diagnostic Commands

<code>kubectl get pods -n <namespace></code>	Confirm the pod is in CrashLoopBackOff and check the RESTARTS count.
<code>kubectl describe pod <pod> -n <namespace></code>	Look at the Events section for 'Back-off restarting failed container' and check Last State > Exit Code / Reason (e.g. Error, OOMKilled, Completed).
<code>kubectl logs <pod> -n <namespace></code>	View logs from the current (restarted) container.
<code>kubectl logs <pod> -n <namespace> --previous</code>	View logs from the PREVIOUS crashed instance — this is usually where the real stack trace lives.
<code>kubectl exec -it <pod> -n <namespace> -- sh</code>	If the pod stays up briefly, shell in to inspect files/env vars before it crashes again.

Step-by-Step Resolution

1. Run `kubectl logs <pod> --previous` first — identify the exact exception, missing file, or failed connection string in the stack trace.
2. Check the exit code with `kubectl describe pod <pod>` under Last State. Exit code 1 = app error, 137 = OOM/SIGKILL, 143 = SIGTERM (often a probe or shutdown issue).

3. If it's a config/env issue, verify with **kubectl get deploy <name> -o yaml | grep -A 30 env:** and compare against the values the app expects; fix the ConfigMap/Secret and **kubectl apply -f deployment.yaml**.

4. If it's a dependency issue (DB/API not reachable), add an initContainer that waits for the dependency:

```
initContainers:
- name: wait-for-db
  image: busybox:1.36
  command: ['sh', '-c', 'until nc -z db-service 5432; do echo waiting for db; sleep 2
; done']
```

5. If the liveness probe is killing it too early, increase initialDelaySeconds / failureThreshold:

```
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  initialDelaySeconds: 30
  periodSeconds: 10
  failureThreshold: 5
```

6. After the fix, roll the deployment: **kubectl rollout restart deployment/<name> -n <namespace>** and watch **kubectl rollout status deployment/<name> -n <namespace>**.

7. Confirm stability: **kubectl get pods -w** for a few cycles to ensure RESTARTS stops incrementing.

Prevention Best Practices

- Add proper readiness/liveness probes with realistic initialDelaySeconds so the app has time to boot.
- Set **terminationGracePeriodSeconds** and handle SIGTERM in the app for clean shutdowns.
- Fail fast with clear log messages on missing config, and validate ConfigMaps/Secrets in CI before deploy.
- Use initContainers to gate startup on hard dependencies instead of crash-looping against them.
- Alert on **kube_pod_container_status_restarts_total** rate in Prometheus rather than discovering this manually.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

ImagePullBackOff

Overview

ImagePullBackOff (preceded by ErrImagePull) means the kubelet on the node cannot pull the container image referenced in the pod spec. Kubernetes retries with backoff instead of failing immediately.

Symptoms

- `kubectl get pods` shows STATUS = ImagePullBackOff or ErrImagePull.
- Pod never reaches Running; 0/1 READY indefinitely.

Root Causes

- Wrong image name or tag (typo, or tag was deleted/never pushed).
- Private registry requires authentication and `imagePullSecrets` is missing or wrong.
- Registry is unreachable from the node (network policy, firewall, DNS, or the registry is down).
- Rate limiting from the registry (e.g., Docker Hub anonymous pull limits).
- Image platform/architecture mismatch (e.g., amd64 image on an arm64 node).

Diagnostic Commands

<code>kubectl describe pod <pod> -n <namespace></code>	Events will show the exact error, e.g. 'manifest unknown', 'unauthorized', or 'connection timed out'.
<code>kubectl get pod <pod> -o jsonpath='{.spec.containers[*].image}'</code>	Confirm the exact image string being used, including tag.
<code>docker pull <same-image>:<tag></code>	Reproduce the pull manually from a machine with the same credentials/network path (or use <code>crictl pull</code> on the node).
<code>kubectl get secret <pull-secret> -n <namespace> -o jsonpath='{.data .\.dockerconfigjson}' base64 -d</code>	Verify the <code>imagePullSecret</code> actually contains valid, non-expired registry credentials.

Step-by-Step Resolution

1. Fix a typo/wrong tag: update the manifest with the correct **image: registry/repo:tag** and re-apply:
`kubectl set image deployment/<name> <container>=registry/repo:correct-tag`.
2. Create/refresh a pull secret for a private registry:

```
kubectl create secret docker-registry regcred \
  --docker-server=<registry> --docker-username=<user> \
  --docker-password=<pass> --docker-email=<email> -n <namespace>
```

3. Attach the secret to the pod/serviceaccount:

```
spec:
  imagePullSecrets:
  - name: regcred
(or) kubectl patch serviceaccount default -n <namespace> -p '{"imagePullSecrets": [
{"name": "regcred"}]}'
```

4. If it's registry rate limiting, mirror the image to your own registry (ECR/GCR/ACR/Harbor) and repoint the deployment.
5. If it's a network/firewall issue, confirm egress rules allow the node to reach the registry endpoint (443) and check any NetworkPolicy or proxy config on the node.
6. Re-trigger the pull: **kubectl delete pod <pod> -n <namespace>** (a new pod will retry immediately) then **kubectl get pods -w**.

Prevention Best Practices

- Pin image tags (never use **:latest**) and use immutable digests (**image@sha256:...**) for production.
- Store pull secrets centrally and rotate them before expiry; automate via CI/CD or external-secrets-operator.
- Use a pull-through cache/mirror registry to avoid public rate limits.
- Validate image existence and pull access as a CI/CD pipeline gate before deploying.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Overview

OOMKilled means the Linux kernel's OOM killer terminated the container because it exceeded its memory limit (cgroup limit), or the node itself ran out of memory. It shows up as exit code 137 in pod status.

Symptoms

- `kubectl describe pod` shows 'Last State: Terminated, Reason: OOMKilled, Exit Code: 137'.
- Pod restarts repeatedly, often correlated with traffic spikes or batch jobs.

Root Causes

- Memory limit set too low for real workload (`spec.containers[].resources.limits.memory`).
- Memory leak in the application causing usage to grow unbounded over time.
- Large in-memory caches, big JSON payloads, or JVM heap not sized to the container limit.
- No memory limit set at all, allowing the pod to consume node memory until the node evicts it.

Diagnostic Commands

<code>kubectl describe pod <pod> -n <namespace></code>	Confirm 'OOMKilled' under Last State and note the memory limit in the Containers section.
<code>kubectl top pod <pod> -n <namespace></code>	Check current live memory usage against the configured limit (requires metrics-server).
<code>kubectl get pod <pod> -o jsonpath='{.spec.containers[0].resources}'</code>	See exact requests/limits currently configured.
<code>kubectl logs <pod> --previous</code>	Look for memory-growth warnings or heap errors just before the kill.
<code>kubectl get events -n <namespace> --field-selector reason=OOMKilling</code>	Cluster-level OOM events (also check node-level dmesg via <code>journalctl -k grep -i 'oom'</code> on the node if you have SSH access).

Step-by-Step Resolution

1. Right-size the limit based on observed usage: **`kubectl top pod`** over a normal + peak load window, then set limits ~20-30% above the observed peak.
2. Patch resources directly:

```
resources:
  requests:
    memory: "256Mi"
  limits:
    memory: "512Mi"
kubectl apply -f deployment.yaml
```

3. For JVM apps, explicitly bound the heap to the container limit: **-XX:MaxRAMPercentage=75.0** (Java 11+) instead of relying on defaults.
4. If usage grows over time (leak), capture a heap dump/profile (e.g. **kubectl exec** + language-specific profiler) and fix the leak in code; the resource bump is a stop-gap, not the fix.
5. If it's legitimately a high-memory workload, consider a Vertical Pod Autoscaler in recommendation mode to size requests/limits automatically, or move to a larger node pool.
6. Roll out: **kubectl rollout restart deployment/<name> -n <namespace>** and monitor **kubectl top pod** for headroom under limit.

Prevention Best Practices

- Always set both memory requests and limits; never leave limits unset in production.
- Add memory usage dashboards/alerts at 80% of the limit so you catch pressure before a kill.
- Load-test before shipping to size limits with real data, not guesses.
- For memory-hungry batch jobs, isolate them onto dedicated node pools with appropriate limits.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Pending Pods

Overview

A pod stuck in Pending has been accepted by the API server but the scheduler cannot place it on any node. This is purely a scheduling problem — the container never even starts.

Symptoms

- `kubectl get pods` shows STATUS = Pending indefinitely.
- `kubectl describe pod` Events show 'FailedScheduling'.

Root Causes

- Insufficient CPU/memory on all nodes to satisfy the pod's requests.
- No node matches nodeSelector / nodeAffinity / required labels.
- Node taints without matching tolerations on the pod.
- PVC the pod depends on isn't bound (see issue 9), which also blocks scheduling.
- Cluster autoscaler disabled or maxed out, so no new node can be added.

Diagnostic Commands

<code>kubectl describe pod <pod> -n <namespace></code>	Events show the exact reason, e.g. '0/5 nodes are available: 3 Insufficient cpu, 2 node(s) had taint {...} that the pod didn't tolerate'.
<code>kubectl get nodes -o wide</code>	Check node count, status, and capacity.
<code>kubectl describe nodes grep -A 5 'Allocated resources'</code>	See how much CPU/memory is already reserved on each node.
<code>kubectl get pod <pod> -o yaml grep -A 10 'nodeSelector\ affinity\ tolerations'</code>	Confirm the scheduling constraints on the pod.

Step-by-Step Resolution

1. If it's a resource shortage: reduce the pod's requests to realistic values, or scale the cluster: **`kubectl scale nodepool <name> --replicas=<n>`** (cloud-provider specific) or enable/adjust the Cluster Autoscaler.
2. If it's a nodeSelector/affinity mismatch: **`kubectl get nodes --show-labels`** to confirm the label exists, then fix the selector in the deployment or label the target nodes: **`kubectl label node <node> <key>=<value>`**.
3. If it's a taint issue: either remove the taint **`kubectl taint node <node> <key>:<effect>-`** or add a toleration to the pod spec:

```
tolerations:  
- key: "dedicated"  
  operator: "Equal"  
  value: "gpu"  
  effect: "NoSchedule"
```

4. If it's a PVC dependency, resolve the PVC binding issue first (see issue 9) — the pod will schedule automatically once the PVC is Bound.
5. Re-check placement: **kubectl get pod <pod> -o wide** and **kubectl get events -n <namespace> --sort-by=.lastTimestamp**.

Prevention Best Practices

- Set requests based on real usage, not arbitrary defaults, to avoid over-reserving capacity.
- Enable Cluster Autoscaler (or Karpenter) so capacity shortfalls self-heal.
- Document and enforce consistent node labeling/taint conventions across the cluster.
- Use **kubectl describe node** capacity checks in pre-deploy CI for capacity-sensitive workloads.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Node Not Ready

Overview

A node reporting NotReady means kubelet has stopped sending heartbeats to the API server, or is reporting a condition (memory/disk/PID pressure) that blocks scheduling. Existing pods may keep running but new pods won't schedule there, and after 5 minutes (default) pods get evicted.

Symptoms

- `kubectl get nodes` shows STATUS = NotReady.
- Pods on that node show Unknown or get rescheduled elsewhere after the eviction timeout.

Root Causes

- kubelet process crashed or stopped on the node.
- Disk pressure, memory pressure, or PID pressure condition triggered.
- Network partition between node and control plane (security group / firewall / VPN issue).
- Container runtime (containerd/docker) crashed or is unresponsive.
- Node ran out of resources entirely (kernel OOM at the host level).

Diagnostic Commands

<code>kubectl get nodes -o wide</code>	Confirm which node(s) are NotReady and since when.
<code>kubectl describe node <node></code>	Check Conditions section for MemoryPressure, DiskPressure, PIDPressure, and the last heartbeat time.
<code>kubectl get events --field-selector involvedObject.name=<node></code>	Cluster-level events for that node.
<code>ssh <node> then: systemctl status kubelet</code>	Confirm the kubelet service is actually running on the node (needs node access).
<code>ssh <node> then: journalctl -u kubelet -n 200 --no-pager</code>	Kubelet logs showing the specific failure (cert expiry, disk full, runtime errors).
<code>ssh <node> then: df -h && free -m</code>	Check for disk/memory exhaustion directly on the host.

Step-by-Step Resolution

1. SSH to the node and check kubelet status: `systemctl status kubelet`. If stopped/crashed, restart it: `systemctl restart kubelet` and tail `journalctl -u kubelet -f`.
2. If DiskPressure: free up space (see issue 23) — `docker system prune -af / crictl rmi --prune`, clear old logs in `/var/log`, then confirm condition clears: `kubectl describe node <node> | grep`

Pressure.

3. If it's a container runtime crash: **systemctl restart containerd** (or docker), then restart kubelet.
4. If it's a network/security-group issue, confirm the node can reach the API server: **curl -k https://<api-server>:6443/healthz** from the node, and check security group/firewall rules allow kubelet↔control-plane traffic (typically 6443, 10250).
5. If the node is unrecoverable (cloud VM issue), cordon and drain it, then terminate/replace the underlying instance via your cloud provider or node group:

```
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
```

6. Once healthy, uncordon: **kubectl uncordon <node>** and verify: **kubectl get nodes**.

Prevention Best Practices

- Set up node-level monitoring (node_exporter/Prometheus) for disk, memory, and kubelet health with alerting before NotReady is reached.
- Enable automatic node repair/replacement in managed Kubernetes (GKE node auto-repair, EKS managed node groups with health checks).
- Right-size node pools and set eviction thresholds (**--eviction-hard**) appropriately for your workload profile.
- Rotate kubelet certificates automatically to avoid TLS-expiry-caused NotReady states.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Readiness Probe Failed

Overview

A failing readiness probe doesn't restart the container — it removes the pod's IP from the Service's Endpoints, so no traffic is routed to it. The pod stays Running but shows as NotReady (e.g. 0/1) and effectively becomes invisible to clients.

Symptoms

- `kubectl get pods` shows `READY = 0/1` while `STATUS = Running`.
- `kubectl get endpoints <service>` is missing that pod's IP.
- Intermittent 502/503s from the Service as pods flap in and out of readiness.

Root Causes

- The readiness endpoint returns non-2xx (app-level health check failing, e.g. DB not connected yet).
- Wrong path/port configured in the probe versus what the app actually serves.
- App takes longer to warm up than `initialDelaySeconds` allows.
- Downstream dependency check inside the health endpoint is flapping.

Diagnostic Commands

<code>kubectl describe pod <pod> -n <namespace></code>	Events show 'Readiness probe failed: HTTP probe failed with statuscode: 500' etc.
<code>kubectl get endpoints <service> -n <namespace></code>	Confirm whether the pod IP is present/absent from the Service's endpoint list.
<code>kubectl exec -it <pod> -n <namespace> -- curl -sv localhost:<port>/<probe-path></code>	Manually hit the exact probe endpoint from inside the pod to see the real response/status code.
<code>kubectl get pod <pod> -o yaml grep -A 8 readinessProbe</code>	Confirm the configured path, port, and timing values.

Step-by-Step Resolution

1. Reproduce the probe manually: `kubectl exec -it <pod> -- curl -i localhost:<port>/<path>` and fix whatever the app returns (wrong status code, wrong path, timeout).
2. If the path/port is simply misconfigured, correct the manifest:

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 3
kubectl apply -f deployment.yaml
```

3. If warm-up genuinely takes longer, increase **initialDelaySeconds** and/or **failureThreshold** rather than removing the probe.
4. If the health endpoint checks a flaky downstream (e.g. cache warm-up), separate 'liveness' (is the process alive) from 'readiness' (is it ready to serve) so a slow dependency doesn't cause a restart loop, only a temporary traffic pull.
5. After fixing, confirm the pod rejoins the endpoint list: **kubectl get endpoints <service> -w**.

Prevention Best Practices

- Keep readiness checks lightweight and specific to 'can this pod serve traffic now' — don't overload it with deep dependency checks that cause cascading flapping.
- Set **initialDelaySeconds** based on measured startup time (P95), not guesswork.
- Add a **startupProbe** for slow-starting apps so readiness/liveness don't fire prematurely during boot.
- Alert on **kube_pod_status_ready{condition="false"}** to catch flapping readiness before users notice.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Liveness Probe Failed

Overview

A failing liveness probe tells kubelet the container is unresponsive/deadlocked, so kubelet kills and restarts it. Unlike readiness, this directly causes restarts — a misconfigured liveness probe can turn a healthy app into a restart loop.

Symptoms

- `kubectl describe pod` Events show 'Liveness probe failed' followed by 'Killing container with id ...'.
- `RESTARTS` count increases even though the app is otherwise functioning under load.

Root Causes

- Probe timeout too short for the app under load (slow response, not actually dead).
- Wrong path/port in the probe.
- App becomes briefly unresponsive during GC pauses or heavy CPU work, tripping the probe.
- Probe hitting an endpoint that itself depends on a slow downstream service.

Diagnostic Commands

<code>kubectl describe pod <pod> -n <namespace></code>	Confirm 'Liveness probe failed' events and correlate timing with restarts.
<code>kubectl get pod <pod> -o yaml grep -A 8 livenessProbe</code>	Check configured path/port/timeoutSeconds/periodSeconds.
<code>kubectl top pod <pod> -n <namespace></code>	Check if CPU is maxed at the time of failures (would explain slow probe responses).
<code>kubectl exec -it <pod> -n <namespace> -- curl -w '%{time_total}\n' -o /dev/null -s localhost:<port>/<path></code>	Measure actual response latency of the liveness endpoint.

Step-by-Step Resolution

1. Separate liveness from readiness: liveness should only check 'is the process alive' (e.g. a static 200 endpoint), not downstream dependencies.
2. Increase probe tolerance if the app is legitimately just slow under load, not deadlocked:

```
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  timeoutSeconds: 5
  periodSeconds: 10
  failureThreshold: 3
```

3. If CPU throttling causes slow responses, raise the CPU limit or investigate throttling (see issue 11) instead of loosening the probe indefinitely.
4. Fix the probe target if it's simply wrong (path/port mismatch) and **kubectl apply** the corrected manifest.
5. Add a **startupProbe** to give slow-booting apps a grace period before liveness checks begin:

```
startupProbe:
  httpGet: {path: /healthz, port: 8080}
  failureThreshold: 30
  periodSeconds: 10
```

6. Validate: **kubectl get pods -w** and confirm RESTARTS stabilizes.

Prevention Best Practices

- Never point liveness at an endpoint that checks external dependencies — that turns a downstream outage into a self-inflicted restart storm.
- Base timeout/threshold values on measured P99 latency under peak load, with headroom.
- Use **startupProbe** for any app with variable cold-start time.
- Load test with the probes enabled, not disabled, to catch this before production.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

DNS Resolution Failure

Overview

Pods resolve Service and external names through CoreDNS running in-cluster. When DNS fails, pods can't reach other Services by name (though direct IP or already-cached connections may still work), breaking most in-cluster communication.

Symptoms

- Application errors like 'could not resolve host' or 'Name or service not known'.
- **nslookup/curl** to a Service DNS name times out or returns NXDOMAIN from inside a pod.

Root Causes

- CoreDNS pods are down, crashlooping, or under-resourced.
- CoreDNS Corefile misconfiguration (wrong upstream, bad plugin config).
- kube-dns Service has no healthy endpoints.
- Pod's /etc/resolv.conf misconfigured (wrong dnsPolicy) or ndots causing excessive lookups/timeouts.
- NetworkPolicy blocking egress to CoreDNS on port 53.

Diagnostic Commands

<code>kubectl get pods -n kube-system -l k8s-app=kube-dns</code>	Confirm CoreDNS pods are Running and not crashlooping.
<code>kubectl logs -n kube-system -l k8s-app=kube-dns --tail=100</code>	Look for Corefile errors, upstream timeouts, or throttling messages.
<code>kubectl exec -it <pod> -n <namespace> -- nslookup kubernetes.default</code>	Test in-cluster resolution from an affected pod.
<code>kubectl exec -it <pod> -n <namespace> -- cat /etc/resolv.conf</code>	Confirm nameserver points to the CoreDNS ClusterIP and ndots value.
<code>kubectl get endpoints kube-dns -n kube-system</code>	Confirm the CoreDNS Service actually has healthy pod IPs as endpoints.
<code>kubectl top pod -n kube-system -l k8s-app=kube-dns</code>	Check for CPU/memory pressure on CoreDNS causing dropped queries.

Step-by-Step Resolution

1. If CoreDNS pods are crashlooping, check logs (**kubectl logs -n kube-system <coredns-pod>**) for Corefile syntax errors; fix the ConfigMap:

```
kubectl edit configmap coredns -n kube-system
then: kubectl rollout restart deployment/coredns -n kube-system
```

2. If CoreDNS has no endpoints, check its Deployment status: **kubectl get deploy coredns -n kube-system** and scale/restart as needed: **kubectl scale deploy coredns -n kube-system --replicas=2**.

3. If CoreDNS is under-resourced and dropping queries, raise its resource limits:

```
resources:
  requests: {cpu: 100m, memory: 128Mi}
  limits: {cpu: 500m, memory: 256Mi}
```

4. If a NetworkPolicy is blocking DNS egress, explicitly allow UDP/TCP 53 to kube-system:

```
egress:
- to:
  - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: kube-system}}
  ports:
  - {protocol: UDP, port: 53}
  - {protocol: TCP, port: 53}
```

5. If high ndots (default 5) is causing excessive/slow lookups for external domains, use fully-qualified names (trailing dot) or tune the pod's dnsConfig:

```
dnsConfig:
  options:
  - {name: ndots, value: "2"}
```

6. Re-test: **kubectl exec -it <pod> -- nslookup <service>.<namespace>.svc.cluster.local**.

Prevention Best Practices

- Run CoreDNS with at least 2 replicas and a PodDisruptionBudget for HA.
- Monitor CoreDNS QPS, error rate, and latency via its built-in Prometheus metrics.
- Keep NetworkPolicies explicit about DNS egress whenever default-deny policies are used.
- Avoid unnecessary use of external hostnames with high ndots; consider NodeLocal DNSCache for large clusters to reduce CoreDNS load.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

PVC Pending

Overview

A PersistentVolumeClaim stuck in Pending means no PersistentVolume has been bound to it — either dynamic provisioning failed or no matching static PV exists. Any pod referencing that PVC will also stay Pending.

Symptoms

- `kubectl get pvc` shows STATUS = Pending.
- Dependent pod also stuck Pending with 'unbound PersistentVolumeClaims' in events.

Root Causes

- Referenced StorageClass doesn't exist or has a typo.
- No default StorageClass configured and the PVC didn't specify one.
- Requested storage size/access mode not satisfiable by any available PV or the CSI driver's backend quota.
- CSI provisioner pod is down or misconfigured, so dynamic provisioning never completes.
- Cloud provider quota exhausted (e.g., max EBS volumes per account/zone reached).

Diagnostic Commands

<code>kubectl get pvc <name> -n <namespace></code>	Confirm Pending status and requested StorageClass.
<code>kubectl describe pvc <name> -n <namespace></code>	Events show the exact provisioning failure (e.g. 'no persistent volumes available' or a CSI driver error).
<code>kubectl get storageclass</code>	Confirm the StorageClass referenced actually exists and check which one is marked (default).
<code>kubectl get pods -n kube-system grep -i csi</code>	Check that the CSI provisioner/controller pods are healthy.
<code>kubectl logs -n kube-system -l app=<csi-provisioner-label> --tail=100</code>	CSI provisioner logs often show the true cloud-side error (quota, permissions, AZ mismatch).

Step-by-Step Resolution

1. If StorageClass name is wrong/missing, fix the PVC spec:

```
spec:
  storageClassName: gp3
  accessModes: [ReadWriteOnce]
  resources:
    requests: {storage: 10Gi}
kubectl apply -f pvc.yaml
```

2. If there's no default StorageClass, set one:

```
kubectl patch storageclass <name> -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

3. If the CSI provisioner is down, investigate/restart it: **kubectl rollout restart deployment/<csi-provisioner> -n kube-system**, then re-check **kubectl describe pvc**.
4. If it's a cloud quota/AZ mismatch (pod scheduled in an AZ with no capacity for that volume type), either request a quota increase or add topology-aware provisioning constraints (**volumeBindingMode: WaitForFirstConsumer** in the StorageClass) so the volume is created in the same zone as the pod.
5. For a static PV scenario, create a matching PV manually with the exact accessModes/capacity/storageClassName the PVC expects, then verify binding: **kubectl get pv,pvc**.
6. Once bound, confirm the dependent pod schedules: **kubectl get pods -w**.

Prevention Best Practices

- Set **volumeBindingMode: WaitForFirstConsumer** on StorageClasses to avoid zone-mismatch binding failures.
- Monitor cloud storage quotas and alert before hitting limits.
- Standardize on a single default StorageClass per cluster to avoid PVCs silently referencing a missing class.
- Health-check CSI controller pods as part of cluster monitoring.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Overview

When the underlying volume backing a PVC fills up, the application gets write failures (ENOSPC) even though the pod itself is Running and Ready. Kubernetes won't proactively warn you — this surfaces as application-level errors.

Symptoms

- Application logs show 'No space left on device' or database write errors.
- `kubectl exec df -h` inside the pod shows the mounted volume at or near 100%.

Root Causes

- Log files, temp files, or database growth not being rotated/cleaned.
- PVC sized too small for actual data growth from the start.
- No monitoring/alerting on volume usage, so it silently fills.

Diagnostic Commands

<code>kubectl exec -it <pod> -n <namespace> -- df -h</code>	Check actual filesystem usage percentage on the mounted volume.
<code>kubectl exec -it <pod> -n <namespace> -- du -sh /path/to/mount/* sort -rh head -20</code>	Find which directories/files are consuming the space.
<code>kubectl describe pvc <name> -n <namespace></code>	Confirm current capacity and StorageClass <code>allowVolumeExpansion</code> setting.
<code>kubectl get pvc <name> -n <namespace> -o jsonpath='{.spec.resources.requests.storage}'</code>	Confirm the current provisioned size.

Step-by-Step Resolution

1. Immediate relief: clean up unnecessary files from inside the pod (old logs, temp files, cache) to restore write access: `kubectl exec -it <pod> -- rm -rf /path/to/old-logs/*`.
2. Expand the volume if the StorageClass supports it (check `allowVolumeExpansion: true`), then restart the pod so the filesystem resize is picked up:

```
kubectl patch pvc <name> -n <namespace> -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'
kubectl rollout restart deployment/<name> -n <namespace>
```

3. If the StorageClass doesn't support expansion, provision a larger PVC, migrate data (e.g. via a temporary job that rsyncs old volume → new volume), then repoint the deployment.
4. Set up log rotation/retention inside the app or via a sidecar (e.g. logrotate) so this doesn't recur.
5. Verify space is reclaimed: **kubectl exec -it <pod> -- df -h**.

Prevention Best Practices

- Alert at 80% volume utilization via Prometheus (**kubelet_volume_stats_used_bytes / kubelet_volume_stats_capacity_bytes**).
- Use StorageClasses with **allowVolumeExpansion: true** so growth doesn't require a full migration.
- Enforce log rotation and data retention policies at the application level.
- Size initial PVCs with growth projections, not just current-day usage.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

High CPU Usage

Overview

High CPU usage manifests as increased latency, throttling, or degraded throughput. In Kubernetes this is often compounded by CPU limits causing CFS throttling, which can hurt performance even when the node has spare capacity.

Symptoms

- Elevated p95/p99 latency on the service.
- `kubectl top pod` shows usage at or above the configured CPU limit.
- Throttling visible in `container_cpu_cfs_throttled_periods_total` metrics.

Root Causes

- Genuine traffic spike beyond current replica capacity.
- Inefficient code path (e.g. N+1 queries, unoptimized loop) consuming excess CPU.
- CPU limit set too low, causing throttling even though usage is 'moderate' in absolute terms.
- Missing or misconfigured HPA, so the deployment doesn't scale out under load.

Diagnostic Commands

<code>kubectl top pod -n <namespace> --sort-by=cpu</code>	Identify which pods are consuming the most CPU right now.
<code>kubectl describe pod <pod> -n <namespace> grep -A 3 Limits</code>	Confirm the configured CPU limit for comparison.
<code>kubectl exec -it <pod> -n <namespace> -- top</code>	See what's consuming CPU inside the container itself.
<code>kubectl get hpa -n <namespace></code>	Check whether an HPA exists and whether it's actively scaling.
<code>PromQL: rate(container_cpu_cfs_throttled_periods_total[5m]) / rate(container_cpu_cfs_periods_total[5m])</code>	Quantify how much throttling is occurring (Prometheus/Grafana).

Step-by-Step Resolution

1. If throttling is confirmed, raise the CPU limit (or remove it and rely on requests + node capacity for less-critical workloads):

```
resources:
  requests: {cpu: 500m}
  limits: {cpu: 1000m}
```

2. If it's a genuine traffic spike, scale out replicas immediately: **kubectl scale deployment/<name> --replicas=6 -n <namespace>**, then configure an HPA for future spikes:

```
kubectl autoscale deployment <name> --cpu-percent=70 --min=3 --max=10 -n <namespace>
>
```

3. If it's inefficient code, profile the hot path (language-specific profiler/APM) and optimize the algorithm/query — scaling hardware only masks the underlying cost.
4. Verify metrics-server is healthy so HPA decisions are accurate: **kubectl get apiservice v1beta1.metrics.k8s.io**.
5. After changes, monitor: **kubectl top pod -w** and confirm CPU usage stabilizes below the limit with headroom.

Prevention Best Practices

- Set CPU requests close to real steady-state usage and limits with enough headroom to avoid routine throttling.
- Configure HPA (CPU or custom metrics) on any user-facing service so it scales automatically under load.
- Add APM/profiling to catch regressions in CPU efficiency before they hit production.
- Load test before major releases to validate capacity assumptions.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

High Memory Usage

Overview

Sustained high memory usage degrades performance (GC pressure, swapping if enabled, slow responses) and, left unchecked, leads directly into OOMKilled (issue 3). The distinction here is catching the trend before the kill happens.

Symptoms

- Gradual, steady increase in memory usage over hours/days (classic leak signature) or a sudden spike under load.
- Slower response times as garbage collection runs more frequently.
- `kubectl top pod` trending toward the configured limit.

Root Causes

- Memory leak (unclosed connections, growing caches, unbounded queues).
- Large request/response payloads held in memory simultaneously under concurrent load.
- Runtime GC/heap settings not aligned with the container's memory limit.
- Cache with no eviction policy or TTL.

Diagnostic Commands

<code>kubectl top pod -n <namespace> --sort-by=memory</code>	Rank pods by current memory usage.
<code>PromQL: container_memory_working_set_bytes{pod="<pod>"}</code>	Plot the trend over time to distinguish a leak (steady climb) from a spike (traffic-correlated).
<code>kubectl exec -it <pod> -n <namespace> -- cat /sys/fs/cgroup/memory.max (or memory/memory.limit_in_bytes on cgroup v1)</code>	Confirm what limit the process actually sees.
Language-specific heap dump/profiler (e.g. <code>jmap</code> , <code>pprof</code> , <code>node --inspect</code>)	Capture a heap snapshot to pinpoint what's retaining memory.

Step-by-Step Resolution

1. Confirm leak vs. spike using the Prometheus trend line before doing anything — a leak needs a code fix; a spike needs capacity/scaling.
2. For a genuine spike under load, scale horizontally: **`kubectl scale deployment/<name> --replicas=<n>`** and/or add a memory-based HPA using custom metrics (CPU-only HPA won't react to memory).

3. For a leak, capture a heap dump/profile from a live pod and identify the retaining object graph; patch the code (e.g., close connections, bound cache size, fix event listener leaks) and redeploy.
4. Add explicit cache eviction/TTL where an unbounded cache is the culprit.
5. Align runtime memory settings to the container limit (e.g. Node.js **--max-old-space-size**, JVM **-XX:MaxRAMPercentage**) so the runtime GCs before hitting the hard cgroup limit.
6. Re-check trend after fix: **kubectl top pod -w** and the Prometheus graph should flatten instead of climbing.

Prevention Best Practices

- Alert at 80% of memory limit, not just on OOMKilled after the fact.
- Bake memory leak tests / long-soak tests into the release process for memory-sensitive services.
- Set language-runtime memory flags explicitly rather than trusting defaults with container cgroup limits.
- Review and bound any in-process caches (max size + TTL) as a standard code-review checklist item.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Service Not Reachable

Overview

A Service exists but traffic to it fails or times out. This is almost always a mismatch between the Service's selector and the pod labels, resulting in an empty or wrong Endpoints list, or a wrong targetPort.

Symptoms

- curl to the Service ClusterIP/DNS name times out or connection refused.
- kubectl get endpoints <service> shows <none> or an empty ENDPOINTS column.

Root Causes

- Service selector labels don't match the pod's actual labels.
- Service targetPort doesn't match the container's actual listening port.
- Pods backing the service aren't Ready (readiness probe failing — see issue 6), so they're excluded from Endpoints even though the selector is correct.
- kube-proxy issue on the node (rare, but check if only some nodes are affected).

Diagnostic Commands

<code>kubectl get svc <service> -n <namespace> -o yaml</code>	Check selector and targetPort/port configuration.
<code>kubectl get endpoints <service> -n <namespace></code>	If empty, the selector isn't matching any Ready pod.
<code>kubectl get pods -n <namespace> --show-labels</code>	Compare pod labels directly against the Service selector.
<code>kubectl get pods -n <namespace> -o wide -l <selector-from-svc></code>	Confirm which pods the selector actually matches and their READY state.
<code>kubectl exec -it <another-pod> -- curl -v <service>.<namespace>.svc.cluster.local:<port></code>	Test connectivity from inside the cluster to isolate DNS vs. routing vs. app issues.

Step-by-Step Resolution

1. If the selector doesn't match, fix the Service to match actual pod labels, or relabel the pods (whichever is correct per your intended design):

```
kubectl label pod <pod> app=myapp --overwrite
(or) kubectl edit svc <service> # correct the selector
```

2. If targetPort is wrong, correct it to the container's actual listening port:

```
spec:
  ports:
  - port: 80
    targetPort: 8080    # must match containerPort
kubectl apply -f service.yaml
```

3. If pods are simply not Ready, fix the readiness probe issue first (see issue 6) — the Service will populate Endpoints automatically once pods pass readiness.
4. Confirm the fix: **kubectl get endpoints <service> -w** should show pod IP:port entries appear.
5. Test end-to-end: **kubectl run tmp --rm -it --image=busybox --restart=Never -- wget -q0- http://<service>.<namespace>.svc.cluster.local:<port>.**

Prevention Best Practices

- Use a single consistent label schema (e.g. **app.kubernetes.io/name**) across Deployments and Services to prevent selector drift.
- Add a CI/CD check that validates Service selectors match at least one existing pod template label before deploy.
- Monitor **kube_endpoint_address_available** per Service and alert when it drops to zero.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Ingress Not Working

Overview

Ingress failures happen at any of several layers: the Ingress Controller itself, DNS pointing to the load balancer, TLS/cert handling, or the backend Service/path routing rules. Diagnosing requires walking the request path top-down.

Symptoms

- External requests to the hostname return 404, 502, 503, connection timeout, or SSL errors.
- Requests work via port-forward to the Service directly but not through the Ingress hostname.

Root Causes

- Ingress Controller (nginx/ALB/Traefik) not running or not watching the right IngressClass.
- DNS record for the hostname doesn't point to the Ingress Controller's load balancer.
- Path/host rules in the Ingress resource don't match the request, or backend Service/port is wrong.
- TLS secret missing/invalid for the host (see also issue 15).
- Ingress annotations misconfigured for the specific controller in use.

Diagnostic Commands

<code>kubectl get ingress <name> -n <namespace> -o yaml</code>	Verify host, path rules, backend service/port, and TLS block.
<code>kubectl describe ingress <name> -n <namespace></code>	Events often show backend errors like 'no endpoints available' or cert issues.
<code>kubectl get pods -n <ingress-controller-namespace></code>	Confirm the Ingress Controller pods are Running.
<code>kubectl logs -n <ingress-controller-namespace> <controller-pod> --tail=200</code>	Controller logs show routing decisions and backend errors.
<code>dig <hostname> / nslookup <hostname></code>	Confirm DNS resolves to the correct external IP/load balancer.
<code>curl -vk https://<hostname>/<path></code>	See the exact HTTP response code and TLS handshake behavior.
<code>kubectl get svc <backend-service> -n <namespace> && kubectl get endpoints <backend-service> -n <namespace></code>	Confirm the backend Service referenced by the Ingress has healthy endpoints.

Step-by-Step Resolution

1. Confirm the Ingress Controller is running and watching the correct class:

```
kubectl get ingressclass
spec:
  ingressClassName: nginx    # must match an installed controller
```

2. Fix host/path/backend mismatches directly in the Ingress manifest:

```
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: {name: my-svc, port: {number: 80}}
kubectl apply -f ingress.yaml
```

3. If the backend Service has no endpoints, resolve that first (see issue 13) — Ingress can't route to a Service with zero healthy pods.
4. If DNS is wrong, update the DNS A/CNAME record to point at the Ingress Controller's external IP/hostname: **kubectl get svc -n <ingress-namespace> <controller-service>** to get the current external IP.
5. If TLS is the issue, verify/renew the certificate (see issue 15) and confirm the **tls.secretName** in the Ingress matches an existing, valid Secret.
6. After changes, re-test: **curl -vk https://<hostname>/<path>** and check the controller logs for the request landing correctly.

Prevention Best Practices

- Standardize IngressClass usage across the cluster to avoid controller/resource mismatches.
- Use external-dns to automate DNS record management tied to Ingress/Service changes.
- Add synthetic uptime checks against each public hostname to catch breakage immediately.
- Validate Ingress manifests (**kubectl apply --dry-run=server**) in CI before merging.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Certificate Expired

Overview

Expired TLS certificates break HTTPS to Ingress-exposed services or, at the control-plane level, expired kubelet/API certs can break cluster communication entirely. This entry covers the common Ingress/workload TLS case, using cert-manager as the typical automation tool.

Symptoms

- Browsers/clients show 'NET::ERR_CERT_DATE_INVALID' or SSL handshake failures.
- curl returns 'certificate has expired' errors.

Root Causes

- cert-manager Certificate/Issuer misconfigured or its renewal failed silently (e.g. ACME challenge blocked).
- Manually managed certificate that nobody renewed before expiry.
- ClusterIssuer pointing at a revoked/rate-limited ACME account.
- DNS-01 or HTTP-01 challenge failing due to DNS/Ingress misconfiguration, blocking renewal.

Diagnostic Commands

<code>kubectl get certificate -n <namespace></code>	Check READY status and note the certificate's age vs its intended validity window.
<code>kubectl describe certificate <name> -n <namespace></code>	Events show renewal attempts and failures.
<code>kubectl get certificaterequest,order,challenge -n <namespace></code>	Trace where in the cert-manager issuance pipeline it's stuck (relevant if using ACME/Let's Encrypt).
<code>echo openssl s_client -connect <hostname>:443 -servername <hostname> 2>/dev/null openssl x509 -noout -dates</code>	Directly check the live serving certificate's notBefore/notAfter dates.
<code>kubectl logs -n cert-manager -l app=cert-manager --tail=200</code>	cert-manager controller logs showing the specific renewal error.

Step-by-Step Resolution

1. If cert-manager is installed but renewal failed, force a re-issue by deleting the stale Secret (cert-manager will recreate it):

```
kubectl delete secret <tls-secret-name> -n <namespace>
kubectl describe certificate <name> -n <namespace> # watch it re-issue
```

2. If the ACME challenge is failing (HTTP-01), confirm the Ingress used for the challenge is reachable externally and not blocked by WAF/firewall; for DNS-01, confirm the DNS provider API credentials in the Issuer secret are valid.
3. If using a manually managed cert, renew it with your CA and update the Secret:

```
kubectl create secret tls <tls-secret-name> --cert=fullchain.pem --key=privkey.pem  
-n <namespace> --dry-run=client -o yaml | kubectl apply -f -
```

4. Check the ClusterIssuer/Issuer status for account-level problems (rate limits, revoked account):

```
kubectl describe clusterissuer <name>
```

5. Once reissued, confirm the live cert: **openssl s_client -connect <hostname>:443 -servername <hostname> | openssl x509 -noout -dates** shows a future expiry.
6. Restart the Ingress Controller pods only if it's caching an old cert and not picking up the renewed Secret: **kubectl rollout restart deployment/<ingress-controller> -n <ns>**.

Prevention Best Practices

- Automate certificate issuance/renewal with cert-manager and never manage prod TLS certs manually.
- Alert on certificate expiry at 30/14/7 days out using cert-manager's Prometheus metrics (**certmanager_certificate_expiration_timestamp_seconds**).
- Test ACME challenge paths (HTTP-01 reachability, DNS-01 credentials) as part of cluster health checks.
- Avoid ACME rate limits by using staging issuers during testing and production issuers only for real deploys.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Secrets Not Mounted

Overview

When a pod can't read expected secret values, it's usually a naming/reference mismatch between the Secret object and how the pod consumes it (env var vs. volume mount), or an RBAC restriction, or the Secret simply doesn't exist in that namespace.

Symptoms

- Application errors like 'missing required environment variable' or empty config values.
- Pod fails to start with 'CreateContainerConfigError' referencing a secret.

Root Causes

- Secret name in the pod spec doesn't match the actual Secret name (or wrong namespace — Secrets are namespaced).
- Referenced key inside the Secret doesn't exist (**secretKeyRef.key** typo).
- ServiceAccount lacks RBAC permission to read the Secret (rare, but possible with restrictive RBAC).
- Secret was deleted or never created before the deployment rolled out.

Diagnostic Commands

<code>kubectl describe pod <pod> -n <namespace></code>	Look for 'CreateContainerConfigError' with a message naming the missing secret/key.
<code>kubectl get secret <name> -n <namespace></code>	Confirm the Secret actually exists in the SAME namespace as the pod.
<code>kubectl get secret <name> -n <namespace> -o jsonpath='{.data}' jq 'keys'</code>	List the actual keys inside the Secret to compare against what the pod references.
<code>kubectl get pod <pod> -o yaml grep -B2 -A5 'secretKeyRef\ secretName'</code>	See exactly which secret/key the pod expects.
<code>kubectl auth can-i get secret/<name> --as=system:serviceaccount:<namespace>:<sa-name> -n <namespace></code>	Confirm RBAC allows the pod's ServiceAccount to read the secret (for the Secrets API, not needed for standard env/volume injection, but relevant if the app reads the API directly).

Step-by-Step Resolution

1. Fix a name/namespace mismatch by creating the Secret in the correct namespace or correcting the reference:

```
kubectl create secret generic app-secrets --from-literal=DB_PASSWORD=<value> -n <namespace>
```

2. Fix a wrong key reference in the pod spec:

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: app-secrets
      key: db-password # must exactly match a key in the Secret
```

3. For volume-mounted secrets, confirm the volume and mount are both correctly wired:

```
volumes:
- name: secret-vol
  secret: {secretName: app-secrets}
containers:
- volumeMounts:
  - {name: secret-vol, mountPath: /etc/secrets, readOnly: true}
```

4. If using an external secrets operator (e.g. External Secrets Operator, Sealed Secrets), verify the sync status: **kubectl get externalsecret <name> -n <namespace>** and check its controller logs for sync errors.
5. After fixing, redeploy: **kubectl rollout restart deployment/<name> -n <namespace>** and confirm: **kubectl exec -it <pod> -- printenv | grep DB_PASSWORD** (or check the mounted file).

Prevention Best Practices

- Keep Secret and Deployment manifests in the same namespace-scoped Helm chart/kustomize overlay so they're never applied out of sync.
- Use an external-secrets/sealed-secrets pipeline with validation so missing keys fail CI, not production.
- Add a pre-deploy check (**kubectl get secret**) in CI/CD to confirm referenced secrets exist before rollout.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

ConfigMap Changes Not Reflected

Overview

Kubernetes does not restart pods automatically when a mounted ConfigMap changes. Volume-mounted ConfigMaps do eventually sync (via kubelet's periodic sync, default ~1 minute, subject to caching), but env-var-sourced ConfigMap values are injected only once at pod start and NEVER update without a pod restart.

Symptoms

- Application keeps behaving per the old config after **kubectl apply** on the ConfigMap.
- Env-var-based config never updates no matter how long you wait.

Root Causes

- ConfigMap consumed via env vars (**envFrom/valueFrom.configMapKeyRef**), which are only read at container start.
- Volume-mounted ConfigMap update hasn't propagated yet (sync delay) or the app caches the file content in memory and doesn't re-read it.
- subPath volume mounts, which do NOT receive live updates from ConfigMap changes at all.

Diagnostic Commands

<code>kubectl get configmap <name> -n <namespace> -o yaml</code>	Confirm the ConfigMap actually contains the new/expected values.
<code>kubectl exec -it <pod> -n <namespace> -- cat /path/to/mounted/config</code>	Check whether the mounted file inside the pod reflects the new content yet.
<code>kubectl get pod <pod> -o yaml grep -B3 -A5 'configMapKeyRef\ envFrom\ subPath'</code>	Determine HOW the pod consumes the ConfigMap — this determines the fix.
<code>kubectl get pod <pod> -o jsonpath='{.status.startTime}'</code>	Compare pod start time to the ConfigMap's last-modified time to see if a restart already happened.

Step-by-Step Resolution

1. If consumed via env vars, a restart is mandatory — there is no live-reload path:

```
kubectl rollout restart deployment/<name> -n <namespace>
```

2. If consumed via a subPath volume mount, switch to mounting the whole ConfigMap as a directory (subPath breaks live updates by design):

```
volumeMounts:
- {name: config-vol, mountPath: /etc/app-config} # no subPath
volumes:
- name: config-vol
  configMap: {name: app-config}
```

3. If it's a plain volume mount and still stale, wait for kubelet's sync interval (~60s) and confirm the app watches/re-reads the file rather than caching it once at startup; if the app doesn't hot-reload, restart it anyway:

kubectl rollout restart deployment/<name>.

4. For guaranteed rollout on every config change, add a checksum annotation that changes whenever the ConfigMap changes, forcing a new pod hash (common Helm pattern):

```
spec:
  template:
    metadata:
      annotations:
        checksum/config: '{{ include (print $.Template.BasePath "/configmap.yaml")
. | sha256sum }}'
```

5. Verify: **kubectl exec -it <new-pod> -- cat /etc/app-config/<file>** matches the updated ConfigMap.

Prevention Best Practices

- Prefer volume mounts over env vars for config that needs to change without redeploying application code.
- Never use subPath for ConfigMaps/Secrets that need live updates.
- Adopt the checksum-annotation pattern (Helm) or a tool like Reloader (stakater/Reloader) to auto-restart deployments when their ConfigMap/Secret changes.
- Document clearly for each service whether its config is env-based (restart required) or file-based (live-reload possible).

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Deployment Stuck

Overview

A rollout that hangs mid-way means new ReplicaSet pods aren't becoming Ready fast enough (or at all) to satisfy the Deployment's rolling update strategy (maxUnavailable/maxSurge), so old pods are never fully replaced.

Symptoms

- `kubectl rollout status deployment/<name>` hangs with 'Waiting for deployment ... to progress'.
- `kubectl get rs` shows both old and new ReplicaSets partially scaled, neither fully up nor down.

Root Causes

- New pods failing readiness probes (config/app issue in the new version).
- New pods stuck Pending due to insufficient resources for the surge capacity.
- ImagePullBackOff on the new image tag.
- PodDisruptionBudget preventing old pods from terminating.
- **progressDeadlineSeconds** reached without the rollout completing.

Diagnostic Commands

<code>kubectl rollout status deployment/<name> -n <namespace></code>	Confirm it's genuinely stuck, not just slow.
<code>kubectl get rs -n <namespace> -l app=<name></code>	See old vs new ReplicaSet desired/current/ready counts.
<code>kubectl get pods -n <namespace> -l app=<name> --sort-by=.metadata.creationTimestamp</code>	Identify the new pods and their status (Pending/CrashLoopBackOff/ImagePullBackOff).
<code>kubectl describe pod <new-pod> -n <namespace></code>	Root-cause events for why the new pod isn't becoming Ready.
<code>kubectl describe deployment <name> -n <namespace></code>	Check Conditions for 'ProgressDeadlineExceeded' and the configured strategy (maxUnavailable/maxSurge).
<code>kubectl get pdb -n <namespace></code>	Confirm no PodDisruptionBudget is blocking old pod termination.

Step-by-Step Resolution

1. Identify the failing new pod's root cause via **`kubectl describe pod/logs`** and fix it using the relevant playbook above (CrashLoopBackOff, ImagePullBackOff, Pending, etc.) — the Deployment will resume automatically once new pods pass readiness.

2. If the new version is simply broken, roll back immediately to restore service:

```
kubectl rollout undo deployment/<name> -n <namespace>
kubectl rollout status deployment/<name> -n <namespace>
```

3. If it's a capacity issue during surge, temporarily raise **maxSurge**/reduce it, or free up cluster capacity, then retry:

```
spec:
  strategy:
    rollingUpdate: {maxUnavailable: 1, maxSurge: 1}
```

4. If a PDB is blocking termination of old pods, review whether **minAvailable**/**maxUnavailable** on the PDB is compatible with the rollout — temporarily adjust only if safe: **kubectl edit pdb <name>**.
5. Check a specific rollout history to compare against a known-good revision: **kubectl rollout history deployment/<name>** and **kubectl rollout undo deployment/<name> --to-revision=<n>**.

Prevention Best Practices

- Set **progressDeadlineSeconds** and alert on Deployment 'ProgressDeadlineExceeded' conditions so stuck rollouts page someone quickly.
- Canary/staged rollouts (Argo Rollouts, Flagger) catch bad releases before they affect all replicas.
- Keep readiness probes accurate so the rollout mechanism gets a true signal of pod health.
- Test new images/configs in a staging environment identical to production before rolling to prod.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

NetworkPolicy Blocking Traffic

Overview

NetworkPolicies are additive-deny: once any policy selects a pod for a given direction (ingress/egress), all traffic in that direction is denied unless explicitly allowed by a rule. A newly applied or overly-restrictive policy can silently cut off legitimate traffic.

Symptoms

- Pod-to-pod or pod-to-external connections that worked before suddenly time out.
- Only some services are affected — typically ones whose pods match a policy's podSelector.

Root Causes

- A default-deny policy applied without corresponding allow rules for legitimate traffic.
- podSelector/namespaceSelector in the policy doesn't match the actual source/destination pod labels.
- Missing egress rule for DNS (port 53) when a default-deny egress policy is in place (see issue 8).
- Policy applies to the wrong namespace or the CNI plugin doesn't support NetworkPolicy enforcement at all (e.g., some basic CNI setups).

Diagnostic Commands

<code>kubectl get networkpolicy -n <namespace></code>	List all policies in the namespace to see what's in effect.
<code>kubectl describe networkpolicy <name> -n <namespace></code>	Inspect exact podSelector, and Ingress/Egress rules.
<code>kubectl get pods -n <namespace> --show-labels</code>	Confirm source and destination pod labels against policy selectors.
<code>kubectl exec -it <source-pod> -n <namespace> -- nc -zv <dest-pod-ip> <port></code>	Directly test whether traffic is blocked at the network layer (timeout = blocked, refused = policy not the issue).
<code>kubectl get pods -n kube-system -l k8s-app=<cni-name></code>	Confirm the CNI plugin (Calico/Cilium/etc.) is running and healthy, since it's what actually enforces the policy.

Step-by-Step Resolution

1. Identify exactly which policy is scoping the affected pod: `kubectl get networkpolicy -n <namespace> -o yaml` and check each policy's podSelector against the pod's labels.
2. Add an explicit allow rule for the legitimate traffic rather than removing the policy wholesale:

```
spec:
  podSelector: {matchLabels: {app: backend}}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector: {matchLabels: {app: frontend}}
    ports:
    - {protocol: TCP, port: 8080}
```

3. If default-deny egress is blocking DNS, always pair it with an explicit DNS allow rule (see issue 8's egress example) — this is the single most common NetworkPolicy outage.

4. If cross-namespace traffic is blocked, add a namespaceSelector alongside podSelector:

```
ingress:
- from:
  - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: other-ns}}
    podSelector: {matchLabels: {app: client}}
```

5. Apply and re-test immediately: **kubectl apply -f networkpolicy.yaml** then re-run the **nc -zv** test from diagnostics.

Prevention Best Practices

- Roll out default-deny policies gradually, namespace by namespace, with explicit allow rules validated in staging first.
- Maintain a documented traffic matrix (which services talk to which) so policies can be authored correctly instead of by trial and error.
- Always pair egress default-deny with explicit DNS and any required external-API allow rules.
- Use policy visualization/testing tools (e.g. Cilium's **cilium policy trace**, Calico's policy simulation) before applying to production.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

HPA Not Scaling

Overview

The HorizontalPodAutoscaler relies entirely on the metrics pipeline (metrics-server for CPU/memory, or a custom/external metrics adapter for other signals). If that pipeline is broken or the HPA config is off, it silently does nothing — no errors surface to users, just missing autoscaling.

Symptoms

- `kubectl get hpa` shows `TARGETS = <unknown>/<threshold>` instead of an actual percentage.
- Replica count never changes despite load increasing well past the target.

Root Causes

- metrics-server is down, unreachable, or not installed.
- Pods don't have resource requests set — HPA CANNOT calculate a percentage without a request baseline.
- HPA's target metric or thresholds misconfigured.
- minReplicas/maxReplicas bounds already at the ceiling, masking a 'scaling' decision that has nowhere to go.
- Custom metrics adapter (e.g. Prometheus Adapter) misconfigured or its target metric doesn't exist.

Diagnostic Commands

<code>kubectl get hpa <name> -n <namespace></code>	Check TARGETS column — <unknown> means the metrics pipeline is broken.
<code>kubectl describe hpa <name> -n <namespace></code>	Events show the exact reason, e.g. 'failed to get cpu utilization: missing request for cpu'.
<code>kubectl get apiservice v1beta1.metrics.k8s.io</code>	Confirm metrics-server's APIService is Available = True.
<code>kubectl top pods -n <namespace></code>	If this itself fails/errors, metrics-server is the root problem.
<code>kubectl get pod <pod> -o jsonpath='{.spec.containers[0].resources.requests}'</code>	Confirm CPU/memory requests are actually set — required for percentage-based HPA.
<code>kubectl logs -n kube-system -l k8s-app=metrics-server --tail=100</code>	metrics-server logs for scrape failures or TLS errors reaching kubelets.

Step-by-Step Resolution

1. If metrics-server is down or missing, install/restart it:

```
kubectl get deploy metrics-server -n kube-system
kubectl rollout restart deployment/metrics-server -n kube-system
(fresh install: kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/
releases/latest/download/components.yaml)
```

2. If pods lack resource requests, add them — this is mandatory for CPU/memory-based HPA:

```
resources:
  requests: {cpu: 200m, memory: 256Mi}
kubectl apply -f deployment.yaml
```

3. If metrics-server can't reach kubelets (common on self-managed clusters due to TLS), add the standard flag to its args: **--kubelet-insecure-tls** (or properly configure kubelet certs, which is the more secure long-term fix).
4. If bounds are the issue, raise **maxReplicas** if you're actually hitting the ceiling under real load:

```
kubectl patch hpa <name> -n <namespace> -p '{"spec":{"maxReplicas":20}}'
```

5. For custom metrics, verify the Prometheus Adapter is exposing the exact metric name the HPA references: **kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1 | jq**.
6. Confirm it's working: **kubectl get hpa -w** and watch TARGETS populate with real numbers and REPLICAS change under load.

Prevention Best Practices

- Always set resource requests on autoscaled workloads — treat it as a hard requirement, not optional.
- Monitor metrics-server / custom-metrics-adapter health as first-class cluster infrastructure.
- Set maxReplicas with real headroom above expected peak, and alert when REPLICAS sits at maxReplicas for a sustained period (signals you're under-provisioned).
- Test HPA behavior under synthetic load in staging before relying on it in production.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

etcd Performance Issue

Overview

etcd is the cluster's single source of truth; when it's slow, every API server operation (kubectl commands, scheduling, controller reconciliation) slows down with it. This is a control-plane issue, most common in large or long-running clusters with excessive object churn.

Symptoms

- kubectl commands (get/apply/delete) noticeably slow across the whole cluster.
- Controller loops (Deployments, HPA, etc.) lag behind expected reconciliation time.
- etcd leader elections happening more frequently than expected.

Root Causes

- etcd database size grown too large (default alarm at 2GB, hard quota typically 8GB) without compaction/defragmentation.
- Disk I/O latency too high for etcd's fsync requirements (etcd needs low-latency SSD storage).
- Excessive write load from high object churn (e.g., very frequent Event or CRD writes, huge numbers of ConfigMaps/Secrets).
- Network latency between etcd cluster members (multi-AZ etcd with poor connectivity).

Diagnostic Commands

<code>etcdctl endpoint health --cluster -w table</code>	Quick health/latency check across all etcd members (run with proper --cacert/--cert/--key or via the control-plane's etcd pod).
<code>etcdctl endpoint status --cluster -w table</code>	Shows DB size, whether an alarm is raised (NOSPACE), and the current leader.
<code>etcdctl alarm list</code>	Confirm if a NOSPACE or CORRUPT alarm is currently active, which will block writes entirely.
<code>kubectl -n kube-system exec etcd-<node> -- etcdctl --endpoints=https://127.0.0.1:2379 endpoint status -w table</code>	Run status checks from inside the etcd pod on a kubeadm-style cluster.
<code>PromQL: histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m]))</code>	fsync latency — anything consistently above ~10ms indicates disk I/O is the bottleneck.
<code>PromQL: etcd_mvcc_db_total_size_in_bytes</code>	Track DB size growth over time to catch bloat before hitting the quota.

Step-by-Step Resolution

1. If a NOSPACE alarm is active, compact and defragment to reclaim space:

```
etcdctl compact $(etcdctl endpoint status --write-out='json' | jq -r '.[0].Status.header.revision')
etcdctl defrag --cluster
etcdctl alarm disarm
```

2. If DB size is chronically large, review object churn — excessive Events (**kubectl get events -A | wc -l**) or high-frequency CRD writes are common culprits; reduce Event TTL retention or the offending controller's write frequency.
3. If disk I/O is the bottleneck, move etcd onto dedicated, low-latency SSD storage (this often requires provisioning new control-plane nodes with proper storage and migrating etcd members one at a time).
4. If it's cross-AZ network latency, co-locate etcd members in the same low-latency network segment/AZ where your SLA allows, or tune **--heartbeat-interval** / **--election-timeout** conservatively per etcd's own tuning guidance.
5. Always take a fresh snapshot before any etcd maintenance: **etcdctl snapshot save backup.db**.
6. After remediation, re-check: **etcdctl endpoint status --cluster -w table** for reduced DB size and healthy leader/latency.

Prevention Best Practices

- Run etcd on dedicated, fast (NVMe/SSD) storage separate from other workloads.
- Schedule regular **etcdctl defrag** during low-traffic maintenance windows on managed-yourself clusters.
- Monitor **etcd_mvcc_db_total_size_in_bytes** and fsync latency continuously, alerting well before quota limits.
- Take automated periodic etcd snapshots as part of disaster-recovery posture, independent of performance work.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

API Server Slow

Overview

A slow API server delays every `kubectl` command and every controller's reconciliation loop. Root causes usually trace back to etcd latency, excessive client request volume, or resource pressure on the API server pods themselves.

Symptoms

- `kubectl` commands hang or take several seconds to return.
- Controllers (Deployment, HPA, etc.) show delayed reactions to changes.
- `kubectl get --raw /metrics` shows elevated request latency histograms.

Root Causes

- Underlying etcd is slow (see issue 21) — API server latency often mirrors etcd latency directly.
- Excessive request volume from a misbehaving client (a controller/CI job hammering the API in a tight loop, or **kubectl get** with no field selectors pulling huge lists repeatedly).
- API server pods under-resourced (CPU/memory) for the cluster's actual object count and request rate.
- Too many CRDs/webhooks adding latency to every request (validating/mutating admission webhooks that are themselves slow).

Diagnostic Commands

<code>kubectl get --raw /metrics grep apiserver_request_duration_seconds</code>	Raw latency histogram per verb/resource straight from the API server.
<code>kubectl top pod -n kube-system -l component=kube-apiserver</code>	CPU/memory usage of API server pods (self-managed control plane).
<code>kubectl get --raw '/metrics' grep apiserver_current_inflight_requests</code>	Check if the server is near its max concurrent request limits (mutating/readonly).
<code>kubectl get events -A --sort-by=.lastTimestamp tail -50</code>	Look for unusual volume/patterns suggesting a runaway client.
<code>kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations</code>	List active admission webhooks — a slow webhook adds latency to EVERY matching request.
<code>kubectl get --raw /metrics grep webhook_admission_duration_seconds</code>	Identify which specific webhook is adding the most latency.

Step-by-Step Resolution

1. If etcd is the root cause, fix etcd performance first (see issue 21) — API server latency will typically resolve alongside it.
2. Identify and throttle/fix a runaway client: check **apiserver_request_total** broken down by **user_agent/client** in Prometheus, and fix the offending controller/script (add backoff, use watches instead of polling, add field/label selectors to narrow list calls).
3. If API server pods are resource-constrained (self-managed clusters), increase their CPU/memory allocation in the static pod manifest or control-plane node group, then restart:

```
(edit /etc/kubernetes/manifests/kube-apiserver.yaml on the control-plane node to add just resources, kubelet will auto-restart the static pod)
```

4. If a specific webhook is slow, fix or scale that webhook's backend, or mark it non-blocking where safe (**failurePolicy: Ignore** only if appropriate, and reduce its **rules** scope so it only intercepts the resources it truly needs).
5. For managed Kubernetes (EKS/GKE/AKS), open a support case citing the specific latency metrics if the control plane itself (which you don't operate) is the bottleneck.
6. Re-check after remediation: **kubectl get --raw /metrics | grep apiserver_request_duration_seconds** for improved p99 latency, and confirm **kubectl** commands feel responsive again.

Prevention Best Practices

- Monitor **apiserver_request_duration_seconds** and **apiserver_current_inflight_requests** continuously with alerting thresholds.
- Enforce client-side best practices (watches over polling, proper list pagination, label/field selectors) across internal tooling and CI/CD.
- Keep admission webhooks fast, narrowly scoped, and highly available; treat webhook latency as a first-class SLO.
- Right-size and horizontally scale API server replicas ahead of cluster/object-count growth, not reactively.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Worker Node Disk Full

Overview

When a node's disk (usually the container runtime's image/log storage) fills up, kubelet trips DiskPressure and starts evicting pods to reclaim space — often taking healthy workloads down as collateral damage.

Symptoms

- Pods on the node show STATUS = Evicted.
- `kubectl describe node` shows condition DiskPressure = True.
- New pods can't be scheduled to that node.

Root Causes

- Accumulated unused container images taking up disk space.
- Application or container runtime logs not rotated, growing unbounded in `/var/log`.
- emptyDir volumes or ephemeral storage from pods consuming more than expected.
- Docker/containerd overlay filesystem bloat from many image layers.

Diagnostic Commands

<code>kubectl describe node <node> grep -A 5 Conditions</code>	Confirm DiskPressure = True and since when.
<code>kubectl get pods -A -o wide --field-selector spec.nodeName=<node> grep Evicted</code>	See which pods were evicted due to the pressure.
<code>On the node: df -h</code>	Identify which filesystem/mount is actually full.
<code>On the node: du -sh /var/lib/docker/* 2>/dev/null (or /var/lib/containerd/*) sort -rh head</code>	Find what's consuming space in the runtime's data directory.
<code>On the node: du -sh /var/log/* sort -rh head</code>	Check for oversized/unrotated log files.
<code>crictl images (or docker images)</code>	List cached container images to identify prune candidates.

Step-by-Step Resolution

1. Immediate relief — prune unused images and stopped containers on the node:

```
crictl rmi --prune          # containerd
(or) docker system prune -af --volumes # docker
```

2. Clear oversized/rotted logs if log rotation isn't configured: `truncate -s 0 /var/log/<big-file>.log` (for actively-written files) and configure logrotate going forward.
3. Confirm kubelet's garbage collection thresholds are reasonable (defaults are usually fine, but verify):

```
--image-gc-high-threshold=85
--image-gc-low-threshold=80
--eviction-hard=nodefs.available<10%,imagefs.available<15%
```

4. Once space is reclaimed, confirm the condition clears: `kubectl describe node <node> | grep DiskPressure` should show False.
5. Uncordon the node if it was auto-cordoned: `kubectl uncordon <node>`.
6. For repeat offenders, resize the node's root/data volume (cloud provider specific) rather than only fire-fighting.

Prevention Best Practices

- Monitor node disk usage (`node_filesystem_avail_bytes`) with alerts at 80%/90% thresholds, well before kubelet's own eviction threshold.
- Enforce ephemeral-storage requests/limits on pods that write significant temp data.
- Configure log rotation (logrotate or a logging sidecar shipping to a central store) so logs never accumulate unbounded on the node.
- Regularly prune unused images via a scheduled job or rely on kubelet's automatic image GC with sane thresholds.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Failed Rolling Update

Overview

A failed rolling update means the new ReplicaSet's pods come up but fail health checks or crash, so Kubernetes halts the rollout (per the configured strategy) rather than replacing all healthy old pods with a broken version.

Symptoms

- `kubectl rollout status` shows the update not completing, with new pods `CrashLoopBackOff` or failing readiness.
- `kubectl get rs` shows the new ReplicaSet stuck below its desired replica count.

Root Causes

- New image/build has a bug causing crashes or failed health checks.
- New config (env vars, ConfigMap, Secret) referenced by the new version is missing or wrong.
- Breaking schema/API change not yet supported by a dependency (e.g., DB migration not applied yet).
- Resource requests changed in the new version and now don't fit available capacity.

Diagnostic Commands

<code>kubectl rollout status deployment/<name> -n <namespace></code>	Confirm the rollout is stalled, not just slow.
<code>kubectl rollout history deployment/<name> -n <namespace></code>	See revision history to identify the last known-good revision.
<code>kubectl describe pod <new-pod> -n <namespace></code>	Root-cause the specific failure (crash, probe failure, scheduling) of the new version's pods.
<code>kubectl logs <new-pod> -n <namespace> --previous</code>	Application-level error from the new build.
<code>kubectl diff -f deployment.yaml</code>	Compare the new manifest against what's live to catch unintended changes.

Step-by-Step Resolution

1. Roll back immediately to restore service while you investigate — this is almost always the right first move in production:

```
kubectl rollout undo deployment/<name> -n <namespace>
kubectl rollout status deployment/<name> -n <namespace>
```

2. To roll back to a specific known-good revision instead of just the previous one:

```
kubectl rollout undo deployment/<name> --to-revision=<n> -n <namespace>
```

3. Diagnose the new version offline (in staging) using the exact error captured from **kubectl logs --previous** — fix the code/config bug there, not in prod.
4. If it's a missing dependency (e.g. DB migration), run the required migration/setup step BEFORE re-attempting the deploy, and consider using an initContainer or a Kubernetes Job as an explicit pre-deploy step.
5. If it's a resource-fit issue, adjust requests/limits to match actual available node capacity, or scale the cluster before retrying.
6. Once fixed and validated in staging, redeploy: **kubectl set image deployment/<name> <container>=<fixed-image> -n <namespace>** and watch **kubectl rollout status** to confirm success this time.

Prevention Best Practices

- Use canary or blue/green deployment strategies (Argo Rollouts, Flagger) to catch bad releases on a small percentage of traffic before full rollout.
- Gate deploys on passing staging smoke tests identical to what production readiness probes check.
- Run DB migrations as a separate, ordered step (Job/initContainer) before the app version that depends on the new schema goes live.
- Keep **kubectl rollout history** meaningful by using **--record** or CI-set change-cause annotations for fast identification of good revisions.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.

Namespace Stuck in Terminating

Overview

A namespace stuck in Terminating almost always means one or more resources inside it have a finalizer that never completes, blocking the Kubernetes garbage collector from finishing namespace deletion.

Symptoms

- `kubectl get ns` shows STATUS = Terminating indefinitely, sometimes for hours/days.
- `kubectl delete ns <name>` hangs or returns immediately but the namespace never disappears.

Root Causes

- A custom resource (CRD instance) has a finalizer whose owning controller is no longer running to remove it.
- A resource's finalizer references a webhook/controller that's since been deleted or is unreachable.
- Kubernetes API discovery issue where the namespace controller can't enumerate all resource types to check for remaining objects.

Diagnostic Commands

<code>kubectl get ns <name> -o json jq '.status'</code>	Confirm Terminating status and check <code>.status.conditions</code> for specific blocking reasons.
<code>kubectl api-resources --verbs=list --namespaced -o name xargs -n 1 kubectl get --show-kind --ignore-not-found -n <name></code>	Enumerate every resource type still present in the namespace — the culprit is usually a leftover CRD instance.
<code>kubectl get ns <name> -o json jq '.spec.finalizers, .metadata.finalizers'</code>	Show which finalizers are set at the namespace level.
<code>kubectl get <resource-type> -n <name> -o json jq '.items[].metadata.finalizers'</code>	For any suspect resource found above, check ITS finalizers specifically.

Step-by-Step Resolution

1. First, try to find and properly resolve the blocking resource/controller rather than force-removing finalizers blindly — identify what's stuck using the enumeration command above.
2. If a specific resource has a stuck finalizer and its owning controller is gone/broken, remove that resource's finalizer directly (only after confirming it's safe — this skips the controller's cleanup logic):

```
kubectl patch <resource-type> <name> -n <namespace> -p '{"metadata":{"finalizers":[]}}' --type=merge
```

3. If the namespace itself is stuck (not an individual resource), edit the namespace's finalizers via the API directly:

```
kubectl get namespace <name> -o json > ns.json
# edit ns.json: remove entries under spec.finalizers (e.g. "kubernetes")
kubectl replace --raw "/api/v1/namespaces/<name>/finalize" -f ns.json
```

4. Faster one-liner version of the same fix:

```
kubectl get namespace <name> -o json | jq '.spec.finalizers = []' | kubectl replace
--raw "/api/v1/namespaces/<name>/finalize" -f -
```

5. Confirm deletion completes: **kubectl get ns <name>** should return 'not found' shortly after.

6. Afterward, investigate WHY the finalizer never cleared (e.g., a CRD's controller was deleted before its custom resources) so you fix the root cause, not just this instance.

Prevention Best Practices

- Always delete custom resources (and let their controllers run cleanup) BEFORE uninstalling the operator/controller that manages them.
- Avoid force-removing finalizers as a routine practice — it bypasses cleanup logic and can leak external resources (cloud load balancers, volumes, DNS records) that the controller would otherwise have removed.
- Monitor for namespaces stuck in Terminating beyond a few minutes and alert, since it usually signals a deeper controller/CRD lifecycle issue.
- When decommissioning an operator, follow its documented uninstall order (CRs → CRDs → controller) precisely.

Found this useful? Connect on LinkedIn or subscribe on YouTube (links on the cover page) for more hands-on Kubernetes & DevOps content.